ECE Capstone Proposal Defense Report

Project Title: [Design and Functional Verification of a 32-bit RISC-V Microprocessor]

Team: [Almas Adilgazyuly | Amir Zhumagulov | Ali Tussupov]

Supervisor: [Daniele Tosi]

Course/Term: ENG 400 Capstone Project, Fall 2025

Date: [28-11-2025]

Prepared by: [Almas Adilgazyuly | Amir Zhumagulov | Ali Tussupov]

Reviewed by: [Galymzhan Nauryzbayev]

TABLE OF CONTENTS

1. Summary **3**
2. Stakeholders, Context, and Constraints **4**
2.1 Stakeholders and Their Roles 4
1. Student Researchers 4
2. Supervisor 4
3. ECE Department 4
4. Industry 4
2.2 Project Context and Environment 4
1. Academic Context 4
2. Technological Context 4
2.3 Limitations of Existing Solutions 5
2.4 Constraints 5
3. Requirements Specification **6**
3.1 Top-Level Requirements (TLRs) 6
3.2 Derived/Interface/Environmental/Regulatory Requirements 7
4. Standards and Compliance Plan **9**
RV32I-ISA-02 9
STD-2 10
IEEE-1364-2005 10
VERILATOR-UG-1.0 10
IEEE-1364-2005 10
5. Concept Exploration and Trade-off Study **11**
Table 5-A. Trade-off Matrix (scores 0–10) 12
6. System Architecture **13**
7. Verification and Validation (V&V) Plan **15**
8. Risk and Contingency Plan **18**
9. Project Plan, Schedule, and Budget **20**
10. Safety, Ethics, Security, and Accessibility **22**
Safety 22
Hazards 22
Controls 22
Evidence 22
Ethics 23
Security 23
11. References **24**
Appendix A. Engineering Standards **26**
Appendix B. Constraints Matrix **27**

1. Summary

This project focuses on designing and verifying a 32-bit RISC-V processor based on the RV32I instruction set. Because RISC-V is open and license-free, it is well suited for learning, experimentation, and hardware development. The goal is to turn theoretical architecture concepts into a fully working and tested processor.

The work begins with a single-cycle CPU and then develops a more efficient multi-cycle version. The multi-cycle design uses an FSM controller, shared hardware components, and basic exception handling through CSRs. Both versions require complete datapath and control logic. To ensure correctness, a verification environment will be set up using Verilator, the RISC-V GNU toolchain, Python automation, and official rv32ui/rv32mi compliance tests.

The expected outcome is a fully functional RV32I multi-cycle processor that passes standard test suites, along with clear documentation of design decisions and methods. This project supports accessible processor-design education and provides a strong foundation for future improvements such as pipelining, additional instructions, caching, or FPGA implementation.

.

2. Stakeholders, Context, and Constraints

2.1 Stakeholders and Their Roles

1. **Student Researchers**

The designers and verifiers of the 32-bit RISC-V microprocessor. The definition of the requirements, implementation, testing, construction of verification process and documentation, all are the important parts of the responsibility of the researchers. The primary interest lies in achieving functioning project design and correct structure.

2. **Supervisor**

Provided technical oversight, ensured academic quality of the reports and documentation. The methodological and strategical decisions, assessments of research quality and safety, as well as ethical responsibility was delivered by supervisor.

3. **ECE Department**

The department supplies technological resources (computer laboratories) and ensures the project aligns with curriculum outcomes in digital design, computer architecture, and verification engineering through the relevant courses.

4. **Industry**

Industrial processor-design practices (simulation-based verification, compliance tests, resource sharing) are to be used, making the results potentially useful for companies exploring RISC-V-based educational or prototyping cores.

2.2 Project Context and Environment

1. **Academic Context**

The project is developed entirely within the context of the ECE capstone project. The main objective of the project is to learn and understand the processor internals through full RTL implementation and verification.

2. **Technological Context**

The project relies on the following tools and resources:

- **Verilator**: high-speed cycle-accurate simulation, essential for verifying both single-cycle and multi-cycle designs.
- **GNU RISC-V Toolchain**: used to assemble test programs, generate ELF binaries, and extract machine code for memory initialization.

• **Python Automation Scripts**: automate dump-to-hex conversion, test selection, and regression summaries.

• **GTKWave**: waveform debugging for FSM state sequencing, trap handling, and ALU output validation.

• **Yosys**: open-source synthesis for resource estimation (6,447 cells for the multicycle design).

2.3 Limitations of Existing Solutions

Most educational materials stop at single-cycle designs. They do not provide verification infrastructures or real compliance testing. Other than that, Industry-grade formal verification tools are unavailable. Tools like JasperGold or Synopsys Formality are proprietary; thus the project must rely on:

- Verilator
- Simulation-based methodology
- Coverage-oriented test programs

2.4 Constraints

- **Performance Constraints**
  No pipelining; multicycle CPI remains higher than pipelined designs but allows simpler control and higher clock frequency per stage.
- **Compatibility Constraints**
  RTL must remain synthesizable using Yosys and simulatable under Verilator with zero lint warnings.

3. Requirements Specification

3.1 Top-Level Requirements (TLRs)

Table 3-A. System Requirements & Acceptance Criteria.

ID	Requirement (Measurable)	Verify By	Threshold / Acceptance Criteria
REQ - 1	The processor shall correctly execute the full RV32I instruction subset used in the official rv32ui compliance tests.	Verilator simulation, GNU toolchain test programs	100% pass rate across all rv32ui functional tests; no incorrect ALU/control outputs.
REQ - 2	The synthesized multi-cycle CPU shall operate at a minimum stable frequency on the Tang Primer 20K FPGA.	FPGA synthesis/timing report (Gowin IDE)	≥ 20 MHz post-place-and-route timing closure.
REQ - 3	The design shall fit entirely within the logic, BRAM, and DSP resources of the GW2A-18 FPGA.	Gowin resource utilization report	LUT/FF usage ≤ 70% of device capacity; BRAM usage ≤ 50%.
REQ - 4	On-chip memory used for instruction + data storage shall fit within available block RAM.	RTL elaboration, synthesis	≤ 64 KB total BRAM consumption (fits device limit).
REQ - 5	The processor shall provide a means to display register or program-counter output on the Tang Primer 20K LEDs.	On-board test program execution	LED display updates at ≥ 5 Hz for visible feedback.
REQ - 6	The CPU shall initialize PC=0x00000000 and zero out the register file except x0.	Simulation waveform inspection	Startup/reset conditions consistently met across 100% of tests.

REQ - 7	CSR subsystem shall correctly capture mepc, mcause, and jump to mtvec on ECALL/EBREAK.	rv32mi trap-level tests	All trap entries produce correct CSR values; CPU returns via MRET.
REQ - 8	The FPGA build shall provide observable debug signals via logic analyzer or UART.	Synthesis debug probe connection	Must expose at least PC, IR, ALUOut, state register.
REQ - 9	Operation shall remain within the Tang Primer 20K's USB-powered budget.	Power estimation in Gowin IDE	≤ 500 mW total dynamic + static dissipation.

3.2 Derived/Interface/Environmental/Regulatory Requirements.

Derived Requirements.

The processor must support all RV32I instructions used in the rv32ui tests as well as handle branch, memory and CSR-related operations. The processor also should correctly manage exceptions and traps through the CSR registers, including mepc, mcause and mtvec. The control unit must follow the cycle sequence for every instruction and move through fetch, decode, execute, memory access and write-back, for each instruction. The ALU must be able to perform shared functions such as updating the program counter, generating memory addresses, and executing arithmetic and logical operations. During reset, the processor must set the program counter to 0x00000000 and clear all general-purpose registers except for x0.

Interface Requirements.

The processor must provide a simple visual output through the board's 8 user LEDs, typically driven by the program counter or a diagnostic register. It should also expose optional GPIO pins for connecting external peripherals. For debugging, a UART transmitter may be included, operating at standard 115200 baud. The design must remain compatible with the Tang Primer 20K USB-JTAG programming interface provided by the Gowin toolchain.

Environmental Requirements.

The system must operate within the commercial temperature range (0°C to 70°C) and stay within the USB power limit of the board, keeping total consumption below roughly 500 mW. GPIO or

PMOD-connected signals should not exceed moderate switching speeds (around 10 MHz) to maintain signal integrity on the small development board.

Regulatory and Standards Requirements.

All RTL must comply with synthesizable IEEE-1364 Verilog practices, and the processor must behave according to the RV32I specification as validated by standard RISC-V tests. Since the board is USB-powered, the design must respect USB 2.0 power constraints, and normal ESD-safe handling procedures apply during development.

4. Standards and Compliance Plan

There are several standards checking the correctness of processor work and structure. RTL design quality and simulation methods , instruction semantics, encodings, load and store alignments and CSR trap handling are all the clauses that STD-1 is applied to. The rv3lui architectural test reports, signature logs and waveform traces are to confirm the correctness of multi-cycle sequencing.

STD-2 handles the Verilog HDL requirements, FSM correctness and latch-avoidance situations. The correctness is to be confirmed by Yosys synthesis reports, Verilator lint outputs and RTL code checklists with correct HDL structure.

STD-3 will be defining simulation and methodology, including lint requirements, DPI-C trap monitoring and VCD waveform tracing. VCD waveform captures, regression tables and Verialtor build logs are to be used for verification.

Table 4-A. Standards Compliance Matrix.

ID	Standard/Reg	Clause	Requirement Impact	Evidence Type	Due Milestone
STD-1	**RV32I-ISA-02**	Covers instruction behavior, immediates, register semantics, loads/stores, branches, CSR traps	Ensures the processor executes all RV32I instructions correctly; defines correctness of ALU, PC logic, control unit, CSR trap handling	- rv32ui & rv32mi test reports	a0 signature logs, GTKWave state-machine traces
STD-2	**IEEE-1364-2005**	Covers synthesizable constructs, sequential/co	Impacts RTL coding correctness, synthesizabili	Yosys synthesis report	RTL code review checklist

		mbinational logic rules, FSM correctness	ty, stability of FSM, register file integrity, reset logic		
STD-3	**VERILATOR -UG-1.0**	Covers simulation methodology, linting rules, DPI-C usage, trace/assert requirements	Ensures correct simulation behavior, waveform accuracy, ECALL/EBR EAK trap detection, reliable automated regression	VCD waveform captures	summary regression table

5. Concept Exploration and Trade-off Study

Name	Description
Method-1: Single-Cycle RV32I Processor	A single-cycle RV32I processor is designed in a way in which it can complete each subsequent instruction in exactly one clock cycle. The design stays simple overall. It proves easy to check because the control logic works in a purely combinational way.
Method-2: Multi-Cycle RV32I Processor	A multi-cycle RV32I processor is designed to reuse the hardware (ALU) for PC, CSR updates, as well as for effective address calculation, and arithmetic operations. It doesn't finish instructions in a single beat, instead each instruction is spread over 3 to 5 cycles under the control of a compact finite-state machine.
Method-3: 5-Stage Pipelined RV32I Processor	A 5 stage pipelined RV32I processor is designed to overlap instruction execution via IF, ID, EX, MEM, and WB stages. Although this architecture offers near-1 CPI throughput, it also may create hazards that require such operations as forwarding networks, stall logic, and branch handling hardware.

Table 5-A. Trade-off Matrix (scores 0–10)

Criterion (≥5 constraints)	Metric / Target	Weight	Method-1 Single-Cycle	Method-2 Multi-Cycle	Method-3 Pipelined
FPGA Resource Usage	≤70% LUT	0.25	6	9	5
Clock Frequency	≥20 MHz	0.20	5	9	7
Power Consumption	≤500 mW	0.15	6	9	6
Verification Complexity	≤12 weeks	0.25	8	9	4
Scalability	future extensions	0.15	6	8	9
TOTAL		1.00	6.35	8.70	6.00

6. System Architecture

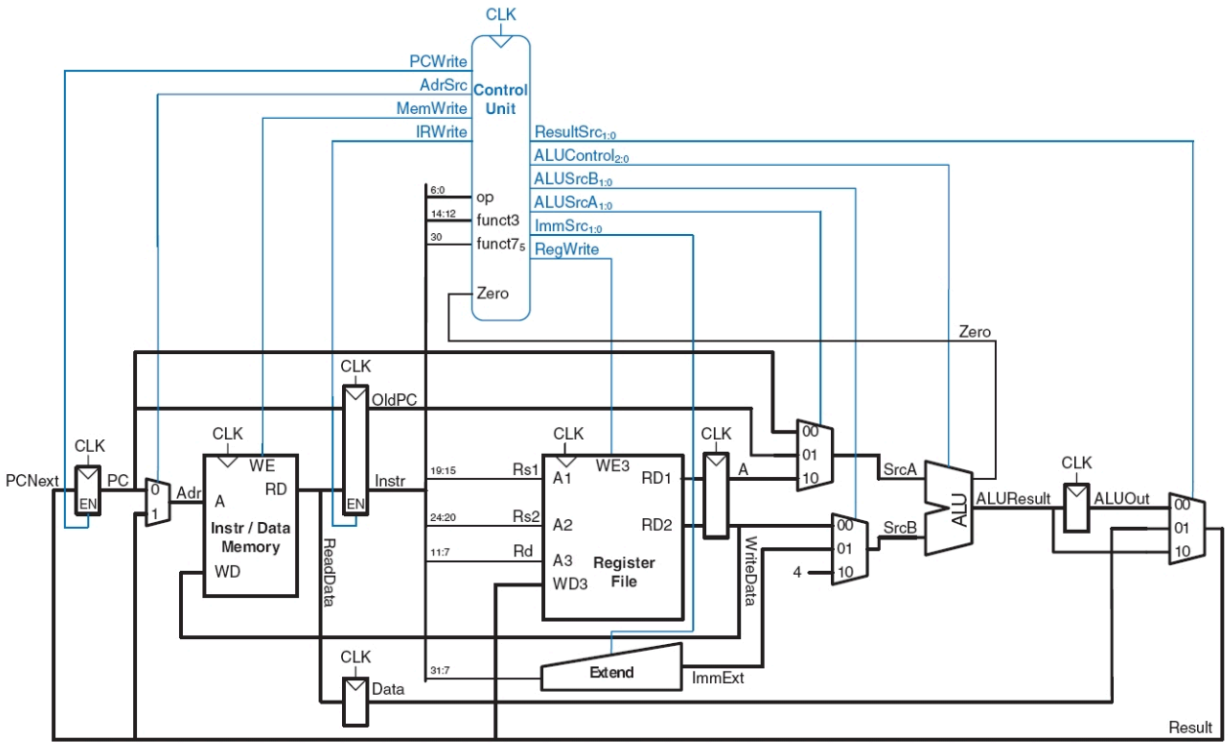


Figure 1 – A reference multi-cycle datapath adapted from Harris and Harris [7].

Table 6-A. Hardware Interface Summary.

Interface	Direction	Protocol	Rate/Timing	Voltage/Power	Notes
RV32I Core ↔ Unified Instr/Data Memory	Core → memory (addr, control), memory → core (read data)	Simple synchronous SRAM-style bus: 32-bit ReadData, 32-bit WriteData, address, MemWrite, IRWrite	1 word access per cycle; target clock 50–100 MHz;	Core supply (FPGA) ~1.0–1.2 V;	Implements instruction fetch and LW/SW; critical to overall CPI and timing; currently modeled as Verilog memory.

RV32I Core ↔ C++ Testbench (Verilator / DPI-C)	Bidirectional (stimulus + observation)	Verilator DPI-C function calls, memory init files, a0/mcause monitoring	“Rate” is simulation timestep; logically one check per instruction or per test end	No physical voltage (simulation only); host PC power	Testbench loads HEX programs into memory, clocks the DUT, and checks pass/fail via a0 and CSR values.
Clock / Reset Generator → RV32I Core	Input to core	Single global clock + synchronous reset	Target clock 50–100 MHz; reset asserted for several cycles at start	Same as core supply (1.0–1.2 V nominal for FPGA)	Defines time base for all multicycle states; worst-case path in ALU/memory must meet this period.
RV32I Core ↔ CSR / Debug Status (to host)	Core → host (status), host → core (initial CSR setup if needed)	Memory-mapped CSR interface plus DPI-C readback of mepc, mcause, mstatus	Updated on trap/return events; checked at end of each test run	Core supply internally; no separate I/O domain in simulation	Used to verify ECALL/EBREAK and misaligned access handling, and to log trap causes in regression summary.

7. Verification and Validation (V&V) Plan

This section describes the verification and validation plan used to ensure the RISC-V RV32I processor meets the requirements listed in Section 3. To validate the design from initial specification to final implementation, we use a combination of cycle-accurate simulation and FPGA-based testing. The first stage involves simulation using Verilator to verify the architecture's functionality. This process verifies the design against the standard RISC-V rv32ui and rv32mi test suites. Additionally, we analyze waveforms to verify the internal state of key components, such as the data path, arithmetic logic unit, register file, state machine, and control and status registers.

To validate the processor in a physical environment, we compared various hardware implementation methods. We considered an application-specific integrated circuit (ASIC) as it provides the most accurate power consumption and clock frequency data. However, this option was rejected during the verification phase due to its high cost, long manufacturing time, and the inability to modify it if errors are discovered after production. We also considered commercial hardware emulators, but they were excluded because their cost exceeded the project budget.

We also evaluated complex programmable logic devices (CPLDs). Although CPLDs offer the advantages of non-volatile memory and a simpler architecture, they were not selected because they typically lack the logic density and on-chip memory resources (e.g., block RAM) required to implement the full 32-bit RISC-V instruction set and its peripherals.

Therefore, we selected a field-programmable gate array (FPGA) as the best platform for hardware validation. FPGAs are superior to other options because they can be reprogrammed, allowing for quick bug fixes without additional manufacturing costs. Furthermore, FPGAs operate significantly faster than software simulations. This speed is critical for testing complex programs and verifying real-world I/O interactions, which are difficult to accurately model in a purely virtual simulation.

Table 7-A. V&V Coverage Matrix.

R ID	Requirement (measurable)	Verification Method	T ID	Std/Clause	Equipment / Tools	Pass/Fail Threshold	Evidence

R1	CPU passes 100% RV32I rv32ui tests	Simulation regression	**T1**	RV32I Unprivileged Spec (instruction behavior)	Verilator, GCC RISC-V toolchain	100% test cases pass	Regression log + VCD
R2	Timing closure at ≥20 MHz	Synthesis + timing analysis	**T2**	Gowin GW2A timing constraints	Gowin IDE, Yosys	Slack ≥ 0 ns at 20 MHz	Timing report
R3	LUT ≤70%, BRAM ≤50%	Resource analysis	**T3**	GW2A-18 resource limits	Gowin IDE	Must meet both limits	Device utilization report
R4	Correct reset (PC=0, registers zeroed except x0)	Simulation waveform inspection	**T4**	IEEE 1364 reset behavior	Verilator, GTKWave	Reset correct in 100% of runs	Reset waveform screenshots
R5	FSM implements correct multi-cycle sequencing	Waveform tracing	**T5**	RV32I control-flow rules	Verilator, GTKWave	All instruction types follow expected state sequence	FSM state trace
R6	Trap handling sets mcause/mepc correctly	rv32mi trap tests	**T6**	RISC-V Privileged Spec (ECALL/EBREAK)	Verilator	Correct CSR values and control flow	Trap test logs

R7	Instruction memory loads correct HEX image	Memory initialization check	**T7**	Gowin memory init format	Gowin IDE, HEX loader	Binary image matches reference	Memory dump comparison
R8	LED output toggles ≥5 Hz	Hardware bench test	**T8**	Tang Primer 20K I/O capability	FPGA board	LEDs visibly toggle ≥5 Hz	Video evidence
R9	Power ≤500 mW	Power estimation	**T9**	USB 2.0 5V/500 mA	Gowin power estimator	Total ≤ 0.5 W	Power report

Methods & Evidence.
Each test (T1–T9) includes a clear procedure defining setup, test steps, measurement, and teardown. Simulation tests will be executed in a controlled environment and the RISC-V toolchain. Waveform validation will be performed to inspect ALU results, CSR updates, state machine transitions, and program counter behavior. FPGA-based tests will use the Tang Primer 20K board and Gowin IDE to evaluate timing, resource utilization, and hardware behavior. All of the evidence will be stored in a structured folder containing logs, VCD files, timing reports, resource summaries, and videos for hardware tests.

Table 7-B. V&V Milestones (align with Project Plan).

Milestone	Week	Owners	Scope	Entry Criteria	Exit Criteria	Artifacts
M1: Simulation Bring-Up	3	Almas	Initial functional checks, reset behavior, PC/IR visibility	RTL compiles; basic test program loads	Reset verified; simple instructions run correctly	M1 report, reset VCD

M2: Early Risk-Reduction Tests	6	Amir, Ali	Validate ALU, branching, CSR, and FSM behavior	RTL modules integrated	High-risk behaviors verified; no blocking issues	Risk memos, simulation logs
M3: Full Compliance & FPGA Review	10	Full Team	Full regression (T1–T9), synthesis and timing	All modules complete	≥90% compliance, timing closed, failures analyzed	V&V pack, regression summary

Early Risk-Retirement (bullets).

- **Functional Risk:** Incorrect CSR or trap handling.
- **Timing Risk:** FSM or ALU creates a long critical path..
- **Resource Risk:** Register file or decoder exceeds LUT budget.

Pass/Fail and Deviations Policy.

Each requirement has one associated quantitative threshold listed in Table 7-A. Any failure generates an incident report containing the requirement ID, test ID, evidence, and preliminary root-cause analysis. After corrections, the failing test must be re-executed until the requirement passes. If RTL or architectural changes occur, all dependent tests are re-run to ensure system-level compliance.

8. Risk and Contingency Plan

Table 8-A. Risk Register.

ID	Risk	L	I	RPN	Mitigation	Responsible	Trigger
RSK-1	RTL module contains hidden logic bug (ALU op, immediate decode, FSM state) revealed	4	5	**20**	Unit-test each module early; add randomized ALU tests; review decoder & FSM logic; enable lint on each commit	Almas	rv32ui test fails or FSM stalls during waveform analysis

	late in regression						
RSK-2	Verification toolchain breaks (Verilator/GNU toolchain version mismatch or update issues)	3	4	**12**	Pin tool versions; keep local mirrored toolchain; maintain backup build environment	Almas	Build errors, failing ELF→HEX conversion, or inconsistent simulation outputs
RSK-3	Incomplete ISA coverage—some instructions or corner cases untested (branches, misaligned loads, CSR traps)	4	4	**16**	Use full rv32ui + rv32mi suite; enable DPI-C trap detection; inspect edge-case waveforms	Ali	Differences between expected a0 signature and actual outputs
RSK-4	Complex FSM or trap-handling logic causes long debugging cycles	3	5	**15**	Add intermediate state logging; simplify control paths; document expected state sequence	Amir	FSM enters invalid state or fails to transition (PC freeze)

L = Likelihood (1 = very low, 5 = very high)
I = Impact (1 = minimal effect, 5 = major delay)
RPN = Risk Priority Number (L × I) → higher = more important to manage early

9. Project Plan, Schedule, and Budget

This project follows a structured timeline that includes key milestones as well as a respectable Gantt chart that represent major state-change events in the development of the RV32I processor

on the Tang Primer 20K FPGA. Each milestone has a clearly defined owner, a measurable entry condition, and an objective exit condition that confirms completion.

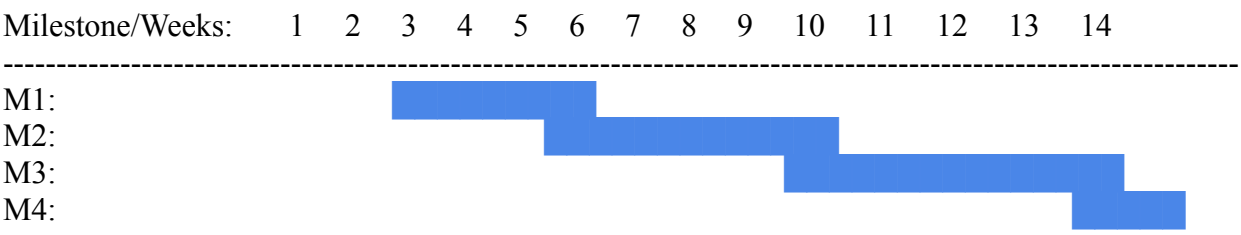


Table 9-A. Milestones.

Milestone	Planned Date	Owner	Entry Criteria	Exit Criteria
M1: Architecture & Requirements Finalization	Week 3	Almas	Draft ISA subset, CPU block diagram, and requirement list prepared	Requirements Spec (Section 3) completed and approved
M2: Initial RTL & Simulation Bring-Up	Week 6	Amir, Ali	Core RTL modules (ALU, RF, FSM skeleton) compile successfully	CPU executes simple instruction sequence in simulation (add, load, branch)
M3: Full Functional Simulation & Compliance Tests, and FPGA Synthesis, Timing, and Resource Validation	Week 10	Team	Integrated datapath and control logic; memory model connected. RTL stable and regression passing	≥95% of RV32I rv32ui tests pass; waveform verification completed. Timing closure at ≥20 MHz; LUT/BRAM usage within GW2A-18 limits
M4: Hardware Demonstration & Final Report	Week 14	Team	FPGA bitstream generated; LED I/O mapped	CPU runs demo program on FPGA; LEDs show live PC or register output;

				final report submitted

A simple bill of materials (BOM) is provided below:

Table 9 - B. Bill of Materials (BOM).

Part No. (PN)	Description	Qty	Unit Cost	Total
FPGA-1	Tang Primer 20K FPGA Development Board (GW2A-18 FPGA)	1	25,000 KZT	25,000 KZT
USB-1	USB-C Cable (programming + power)	2	2,000 KZT	4,000 KZT
ACC-1	Breadboard wires / jumpers for optional GPIO tests	1	2,000 KZT	2,500 KZT
—	*Designing Tools (Verilator, Yosys, GCC-RISC-V)*	—	0 KZT	0 KZT

10. Safety, Ethics, Security, and Accessibility

Safety

The hardware design itself does not drive physical actuators or high-voltage devices during the capstone, but the development environment involves electrical engineering tools, FPGA boards (for potential future extension), and lengthy computation.

Hazards

Incorrect powering of FPGA boards or peripherals may cause short circuits, overheating, or component damage.

Unsafe or corrupted builds from misconfigured Verilator/Yosys paths can create invalid outputs, leading to incorrect results and waste of time.

Controls

- FPGA power rails checked before powering.
- Verified installation paths for Verilator, GCC toolchain, Python scripts, and Yosys.
- Running simulation workloads under controlled CPU limits to avoid overheating laptops/desktops.

Evidence

- Toolchain installation checklist saved in documentation (Verilator/GNU/Yosys versions pinned).
- Supervisor approval required before any FPGA-related work.
- Logs confirming error-free linting and synthesis runs.

Ethics.

Strictly Academic Use:

The design is intended solely for instruction-level educational work, microarchitecture learning, and verification research.

No Deployment in Security-Sensitive Applications:

The core is not intended for cryptographic, surveillance, or safety-critical embedded systems.

Proper Attribution and Open-Source Compliance:

All tools used (Verilator, Yosys, GCC toolchain, RISC-V tests) follow open-source licenses.

Attribution and citations are provided in the report.

Security.

Data Storage:

All project data (RTL, simulation logs, regression summaries) is stored locally on the student's machine.

Accessibility

The final report exported to PDF with selectable text, tagged headings, and searchable content.

11. References

[1] K. Asanovi´c and D. A. Patterson. Instruction sets should be free: The case for RISC-V. Tech. rep. UCB/EECS-2014-146. EECS Department, University of California, Berkeley, 2014. url: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[2] RISC-V International. RISC-V Design Verification Guidelines. Technical Recommendation. Accessed: 5 Oct. 2025. 2024. url: https://riscv.org/technical/specifications/verification-guidelines

[3] R. Patel, S. Wang, and N. Idris. “Energy-Aware Multi-Cycle Microarchitectures for Resource-Constrained RISC-V Designs”. In: ACM Transactions on Architecture and Code Optimization 21.3 (2024). Update with final pagination/DOI once published, pp. 1–25. doi: 10.1145/3651234

[4] M. Garcia, J. Torres, and H. Lee. “Open Verification Pipelines for RISC-V Systems-on-Chip”. In: Proceedings of the 2024 IEEE/ACM International Conference on Computer-Aided Design. Available as RISC-V Summit 2024 white paper; verify final publication details. 2024, pp. 108–115. url: https://riscv. org/wp-content/uploads/2024/07/open-verification-pipelines.pdf

[5] D. A. Patterson and D. R. Ditzel. “The case for the reduced instruction set computer”. In: ACM SIGARCH Computer Architecture News 8.6 (1980), pp. 25– 33. doi: 10.1145/641914.641917.

[6] D. A. Patterson and J. L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition. Morgan Kaufmann, 2017. doi: 10.1016/C2016-0-02534-2

[7] D. Harris and S. Harris. Digital Design and Computer Architecture, RISC-V Edition. Morgan Kaufmann, 2022. doi: 10.1016/C2019-0-03222-2

[8] S. B. Furber. VLSI RISC architecture and organization. CRC Press, 1989. url: https://books.google.com/books?id=Y61_DwAAQBAJ

[9] J. L. Hennessy and D. A. Patterson. Computer Architecture: A Quantitative Approach. 6th. Morgan Kaufmann, 2019. doi: 10.1016/C2017-0-03291-5.

[10] C. N. Keltcher et al. “The AMD Opteron processor for multiprocessor servers”. In: IEEE Micro 23.2 (2003), pp. 66–76. doi: 10.1109/MM.2003.1196116.

[11] E. Rotenberg, S. Bennett, and J. E. Smith. “Trace cache: A low latency approach to high bandwidth instruction fetch”. In: Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture. 1996, pp. 24–34. doi: 10.1109/MICRO.1996.566453.

[12] C. E. Kozyrakis and D. A. Patterson. “A new direction for computer architecture research”. In: Computer 31.11 (1998), pp. 24–32. doi: 10.1109/2.730733.

[13] W. K. Lam. Hardware design verification: Simulation and formal method-based approaches. Prentice Hall, 2005. isbn: 9780131433475.

[14] C. Kern and M. R. Greenstreet. “Formal verification in hardware design: a survey”. In: ACM Transactions on Design Automation of Electronic Systems (TODAES) 4.2 (1999), pp. 123–193. doi: 10.1145/307983.307994.

[15] J. R. Burch et al. “Symbolic model checking: 1020 states and beyond”. In: Information and Computation 98.2 (1992), pp. 142–170. doi: 10.1016 / 0890 - 5401(92)90017-A.

[16] E. M. Clarke, O. Grumberg, and D. E. Long. “Model checking”. In: Logic and Software Engineering. 1996, pp. 141–174. url: https : / / www . cs . cmu . edu / ~emc/papers/LICS90.pdf.

[17] A. Gupta. “Formal hardware verification methods: A survey”. In: Formal Methods in System Design 1.2-3 (1992), pp. 151–238. doi: 10.1007/BF00121239.

[18] C. Wolf. “Yosys – A Practical Open Source Framework for RTL Synthesis”. In: Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip). 2013, pp. 84–87.

Appendix A. Engineering Standards

Table A.1. Engineering Standards.

Standard ID and Title	Clause(s) used	Why applicable?	Verification
RISC-V RV32I ISA Spec	ADD, SUB, AND, OR, XOR, SLT, SLL, SRL, BEQ, LW, SW, JAL	ISA functional correctness for ALU, control, and memory operations (Background §2.1)	Official riscv-tests pass/fail validation; VCD waveform inspection
IEEE 1364 Verilog HDL	Combinational (always @(*)) and sequential coding rules; latch avoidance	Synthesizable and maintainable RTL design; supports educational clarity (Introduction §1.2)	Verilator Wall lint passes with zero warnings; peer code review
Verilator User Guide	Simulation methodology; assert Wall trace command-line options	Cycle-accurate C++ simulation for verification (Background §2.3.1)	Compilation successful; VCD trace generation confirmed; test harness validation
RISC-V GNU Toolchain	Assembly syntax, ELF binary generation, objdump disassembly format	Generate authentic test programs from assembly sources (Methodology Ch. 3)	*converter.py* parses dump files; hex memory images loaded successfully

Appendix B. Constraints Matrix

Table B.1. Multiple-Constraints Matrix.

Constraint	Metric/Target	Rationale	Trade-offs considered	Outcome
Functional correctness (Introduction §1.2)	100% pass on riscv-tests RV32I subset	Ensuring baseline ISA correctness is essential before implementing optional extensions; guarantees a stable foundation for further development.	Full RV32I ALU implemented; extensions (M/A/F/D) deferred	Achieved
Simulation workflow	Single-command build+run; auto-saved results	A simple, automated workflow speeds up iteration, reduces human error, and supports rapid verification during development.	Verilator (fast) vs. commercial tools (Background §2.3.1)	Makefile flow complete; VCD optional
Performance (Introduction §1.2)	Build <30s; runtime <5s; understand single vs. multi-cycle trade-offs	Meeting performance goals keeps test cycles fast, enabling frequent validation while still allowing architectural exploration.	Trace depth vs. speed	Met for core tests; multi-cycle improves efficiency
Ethical (opensource) (Introduction §1.4.1)	RISC-V ISA selection; no licensing fees	Open-source ISA avoids licensing barriers, supports transparency, and aligns with educational and research-focused objectives.	Proprietary ARM vs. open RISC-V	Open ISA chosen; supports educational and global access goals
Professional (verification rigor) (Introduction §1.4.2)	Industry-standard tools: Verilator, riscv-tests	Using established verification tools ensures credibility, reproducibility, and compatibility with standard industry practices.	Simulation vs. formal methods (Background §2.3)	Simulation-based approach validated; reproducible results
Schedule/risk	Ready within term	Narrowing the scope reduces complexity, while a test-first workflow catches issues early, lowering project completion risk.	RV32I scope only; test-first approach	Met; bugs mitigated early